

University of Pisa
Parallel and Distributed Systems

Optimization of Partitioned Hash Join with Duplicates

Module 4 - Report

Gianluca Panzani

May 23, 2026

Contents

1	Implementation	1
1.1	Executable Variants	1
1.2	Parameters	1
2	Distributed Strategy	1
2.1	MPI Redistribution	1
2.2	Local Partition and Join	2
2.3	Measured Phase Times	2
3	Averaged Results	3
3.1	Uniform dataset	3
3.2	Skewed dataset	4
4	Scalability Analysis	4
4.1	Weak Scalability	4
4.2	Strong Scalability	5
5	Correctness Validation	5
6	Conclusions	6

1 Implementation

The implementation keeps the partitioned hash join structure from the previous modules and changes only how the work is placed on the available hardware. Records are generated deterministically, mapped to one of the $P = 256$ partitions. In the distributed versions, MPI is responsible for moving records to the rank that owns their final partition, so each rank can complete its assigned joins independently. The hybrid version adds OpenMP inside each rank to accelerate the local partitioning and joining work after the MPI redistribution has completed.

1.1 Executable Variants

This module extends the partitioned hash join with duplicates to a distributed-memory setting. The sequential `hashjoin_seq` implementation from the previous module is kept as baseline, so every speedup is computed against the sequential execution with the same dataset type.

The OpenMP executable `hashjoin_omp` is the workload-balanced loop implementation from Module 3 and is included in the comparison to show the cost of distributed memory and the potential of hybrid parallelism. In the selected records it uses 32 threads for the partition phase and 32 threads for the join phase, with guided scheduling.

The `hashjoin_mpi` executable follows a pure distributed-memory design. Each MPI rank generates a disjoint slice of the two input relations, redistributes its local records to the ranks that own the corresponding partitions, and then joins only the partitions assigned to it. The `hashjoin_hybrid` executable uses the same ownership and redistribution logic, but the local phases are parallelized with OpenMP. In the measured hybrid grid, the best records use 16 partition threads per rank; the best join-thread count is 16 on uniform data and 4 on both skewed datasets. All selected hybrid records use guided scheduling, chunk size 4, and block size 32768.

1.2 Parameters

The main benchmark campaign uses $N_R = N_S = 50\,000\,000$, $P = 256$, `seed=13`, `max_key=1\,000\,000`, and three datasets: `uniform`, `skewed_90_5`, and `skewed_90_1`. The two skewed datasets concentrate 90% of the records in the keys mapped to a small subset of partitions, respectively about 5% and 1% of the total partition space.

The main comparison reports the best averaged configuration found by the grid search for each execution variant and dataset type. Each reported value is averaged over five repetitions.

2 Distributed Strategy

2.1 MPI Redistribution

The global algorithm follows the same partitioned hash join structure used in the previous modules: each record is assigned to a partition with the same mapping function, records belonging to the same partition are grouped together, and each partition pair (R_p, S_p) is joined independently. In the MPI version, however, partitions are not processed by threads in a shared-memory space, but are assigned to MPI ranks. Therefore, before the local join can start, records must be redistributed so that each rank receives the records of the partitions it owns.

Each rank first generates only a disjoint slice of the global input relations and scans its local records to compute their destination rank. Two ownership policies are implemented:

- **block**: contiguous ranges of partition ids are assigned to ranks;
- **cyclic**: partition p is assigned to rank $p \bmod N_{\text{ranks}}$, where N_{ranks} is the number of MPI processes;

The redistribution phase is performed separately for R and S . For each relation, every rank groups its local records by destination rank. Before exchanging the actual records, each rank must know how

many records it will receive from every other rank, in order to allocate the receive buffer and compute the receive displacements. For this reason, the first communication step uses `MPI_Alltoall`: each rank sends exactly one integer count to every other rank, representing the number of records that will be sent to that destination.

After the count exchange, each rank knows the size of all incoming messages. The actual records are then exchanged with `MPI_Alltoallv`. The variable-size version is required because the number of records sent from rank i to rank j is not constant: it depends on the partition ownership policy and on the input distribution. This is particularly important for skewed datasets, where some partitions receive many more records than others. A regular `MPI_Alltoall` would require all sender-destination pairs to exchange blocks of the same size, forcing padding and wasting memory bandwidth. Instead, `MPI_Alltoallv` allows each pair of ranks to exchange a different number of records, making it the natural choice for this redistribution step.

After redistribution, each rank owns all records belonging to its assigned partitions. The local partition and join phases can then be executed independently on each rank, and the final `join_count`, `checksum1`, and `checksum2` are aggregated with `MPI_Reduce`.

2.2 Local Partition and Join

After the MPI redistribution, each rank has received all records whose partition ids are assigned to it. In other words, a rank does not own the whole dataset, but it owns the complete global content of its assigned partitions: for every partition p owned by rank r , all records belonging to R_p and S_p have been moved to rank r .

The received records are then locally reorganized according to the same $P = 256$ partition layout used by the sequential implementation. This local partitioning step is still useful because the records received through `MPI_Alltoallv` are grouped by source and destination rank, but not necessarily stored in the exact contiguous partition layout required by the local join kernel. Therefore, each rank rebuilds the local partition offsets for its received R and S records. In the hybrid executable, this local partitioning phase is parallelized with the OpenMP blocked histogram/scatter implementation used in the previous module.

Once this local layout has been built, the join phase is completely local. Rank r processes only the partition ids assigned to it. For each owned partition p , it builds a private flat count table from R_p , probes the corresponding records in S_p , and accumulates the local contribution to `join_count`, `checksum1`, and `checksum2`.

In the hybrid executable, the local join phase is further parallelized with OpenMP. Large local partitions can be split into independent probe subranges using the workload-balanced join decomposition introduced in Module 3. Each subrange computes a private partial result, avoiding synchronization in the hot path.

Finally, the local outputs produced by all ranks are summed with `MPI_Reduce`, obtaining the final global `join_count`, `checksum1`, and `checksum2` on rank 0.

2.3 Measured Phase Times

The reported MPI and hybrid partition and join phase times are rank-level maxima. This is intentional because the wall-clock runtime is determined by the slowest rank due to barriers and synchronization. The measured local phases are `partition_time` and `join_time`. The redistribution/residual component includes the `MPI_Alltoall` and `MPI_Alltoallv` exchanges, reductions, barriers, and other synchronization overhead. Therefore, for MPI and hybrid runs, this component approximates the communication cost that must be paid before the local join can complete. The final tables and plots report the average of these per-run rank-level maxima over the five repetitions.

3 Averaged Results

As already mentioned, each average is computed over five repetitions. The got speedup is relative to the sequential baseline with the same dataset type. For sequential and OpenMP records, the MPI-related columns are only used to keep the table format uniform.

Figure 1 and Figure 2 show the phase breakdown and total time for the selected records. Instead, Table 1 reports the best averaged total time and speedup for each execution variant and dataset type, and Table 2 the corresponding phase breakdown. Note that, since Table 1 reports the best configuration found for each execution type, the MPI and hybrid rows are not runs with the same node placement. The exact rank/node configuration is reported explicitly in the table.

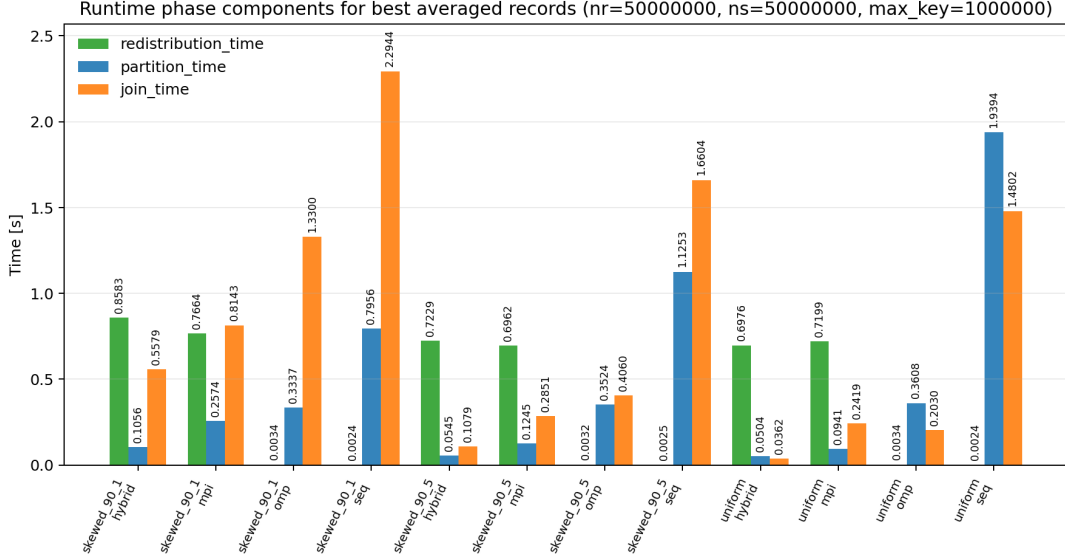


Figure 1: Runtime components for the selected records. The MPI and hybrid residual bars expose the cost of redistribution and synchronization, while the sequential and OpenMP bars are almost entirely local computation.

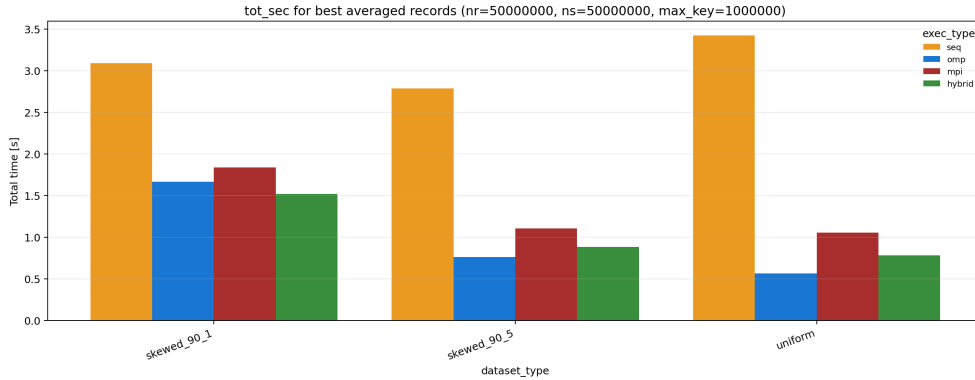


Figure 2: Best averaged total time for the main 50M + 50M benchmark.

3.1 Uniform dataset

On the uniform dataset, OpenMP remains the fastest model. It reaches 0.567 s and $6.03\times$ speedup because the overhead introduced by communication is avoided and the partition/join work is still regular enough for a shared-memory implementation. Pure MPI and hybrid reduce the local computation substantially, but their communication and synchronization costs dominate the final runtime. For example, pure MPI spends only 0.094 s in local partitioning and 0.242 s in local joining, but the redistribution/residual component is 0.720 s, which is already larger than the full OpenMP execution.

The hybrid version makes this trade-off even clearer. On uniform data it reduces local partitioning to 0.050 s and local joining to 0.036 s, but its total time remains 0.784 s because the residual cost is 0.698

s. This means that, for this input size, adding OpenMP inside each MPI rank improves the local phases but cannot remove the communication overhead.

3.2 Skewed dataset

The skewed datasets change the balance. In `skewed_90_5`, OpenMP is still best: it takes 0.762 s, while MPI and hybrid take 1.106 s and 0.885 s. The new hybrid record narrows the gap substantially by using only 4 join threads per rank, reducing the local join to 0.108 s, but the 0.723 s residual component still prevents it from beating OpenMP. In `skewed_90_1`, the join becomes much heavier: OpenMP spends 1.330 s in the join phase, for a total time of 1.667 s, while hybrid reduces the local join to 0.558 s obtaining the best total time among the parallel versions, 1.522 s (Figure 2). Therefore the hybrid approach becomes useful when the local join is expensive enough to compensate for part of the communication cost.

Dataset	Exec.	Nodes	MPI proc.	Ranks/node	P-threads	J-threads	Strategy	Avg total [s]	Speedup
uniform	seq	1	1	1	–	–	–	3.422	1.00×
uniform	omp	1	1	1	32	32	–	0.567	6.03×
uniform	mpi	8	8	1	–	–	cyclic	1.056	3.24×
uniform	hybrid	8	8	1	16	16	cyclic	0.784	4.36×
skewed_90_5	seq	1	1	1	–	–	–	2.788	1.00×
skewed_90_5	omp	1	1	1	32	32	–	0.762	3.66×
skewed_90_5	mpi	1	8	8	–	–	cyclic	1.106	2.52×
skewed_90_5	hybrid	8	8	1	16	4	cyclic	0.885	3.15×
skewed_90_1	seq	1	1	1	–	–	–	3.092	1.00×
skewed_90_1	omp	1	1	1	32	32	–	1.667	1.85×
skewed_90_1	mpi	1	8	8	–	–	cyclic	1.838	1.68×
skewed_90_1	hybrid	8	8	1	16	4	cyclic	1.522	2.03×

Table 1: Best averaged records by execution type.

Dataset	Exec.	Total [s]	Local partition [s]	Local join [s]	Redistribution/residual [s]
uniform	seq	3.422	1.939	1.480	0.002
uniform	omp	0.567	0.361	0.203	0.003
uniform	mpi	1.056	0.094	0.242	0.720
uniform	hybrid	0.784	0.050	0.036	0.698
skewed_90_5	seq	2.788	1.125	1.660	0.003
skewed_90_5	omp	0.762	0.352	0.406	0.003
skewed_90_5	mpi	1.106	0.124	0.285	0.696
skewed_90_5	hybrid	0.885	0.055	0.108	0.723
skewed_90_1	seq	3.092	0.796	2.294	0.002
skewed_90_1	omp	1.667	0.334	1.330	0.003
skewed_90_1	mpi	1.838	0.257	0.814	0.766
skewed_90_1	hybrid	1.522	0.106	0.558	0.858

Table 2: Phase breakdown for the same selected records of Table 1.

4 Scalability Analysis

The scaling campaign uses the pure MPI implementation with one active rank per node. Weak scaling increases the input size proportionally to the number of MPI ranks: both N_R and N_S are set to 10M, 20M, 40M, and 80M records when the number of ranks increases from 1 to 2, 4, and 8, respectively. Strong scaling, instead, keeps $N_R = N_S = 50M$ fixed and varies the allocation from 1 to 8 nodes/ranks.

4.1 Weak Scalability

Weak scaling is best on the uniform dataset. The scaled speedup reaches about 8.1× at 8 nodes, because the input per rank remains almost constant while memory capacity and memory bandwidth grow with

the number of nodes. This is the case where distributed memory is most effective: although records still have to be exchanged through the all-to-all redistribution, the communication cost remains smaller than the gain obtained by using multiple nodes.

As we can observe in Figure 3, the skewed datasets scale much less. With `skewed_90_5`, the 8-node scaled speedup is about $2.2\times$; with `skewed_90_1`, it is about $1.9\times$. Even though the input size per rank is constant, increasing the global input also increases the absolute number of hot records and duplicate matches. This creates both more join work on hot partitions and a less balanced all-to-all exchange.

Compared with the Module 3 OpenMP weak-scaling campaign, MPI is clearly better on uniform data: the previous workload-balanced OpenMP curve stayed near $3\times$ at large thread counts, while the distributed run reaches about $8.1\times$ at 8 nodes. On skewed data the advantage is smaller. For `skewed_90_5`, the MPI result is close to the previous OpenMP peak; for `skewed_90_1`, MPI improves the largest configuration but still remains far from ideal because the hot partitions dominate the computation and communication pattern.

4.2 Strong Scalability

Strong scaling is also good on uniform data. The speedup reaches about $8.1\times$ at 8 nodes, slightly above the ideal 8-rank line in the averaged data. This superlinear behavior is plausible for a memory-bound workload: when the input is split among more ranks, each rank handles smaller buffers and hash tables, which can improve cache behavior and reduce local memory pressure. The result should therefore be interpreted as a combination of parallelism and better locality, not as zero communication cost.

For skewed inputs, strong scaling is limited by hot partitions. `skewed_90_5` and `skewed_90_1` reach about $2.5\times$ and $2.2\times$, respectively, at 8 ranks. Cyclic ownership helps by spreading hot partition ids across ranks, but the pure MPI version still assigns each complete partition to a single rank. If one or a few partitions contain most of the duplicate-heavy work, adding more ranks cannot split that work further.

The comparison with Module 3 explains again the difference between shared-memory and distributed-memory scaling. On uniform data, MPI scales better because it distributes both memory footprint and bandwidth demand across nodes. On skewed data, the previous OpenMP workload-balanced join could split hot local work without paying an all-to-all redistribution cost, while MPI must first move records to partition owners. As a result, MPI is strongest on uniform data and less competitive on skewed inputs unless the local join is heavy enough for the hybrid version to compensate for part of the communication overhead.

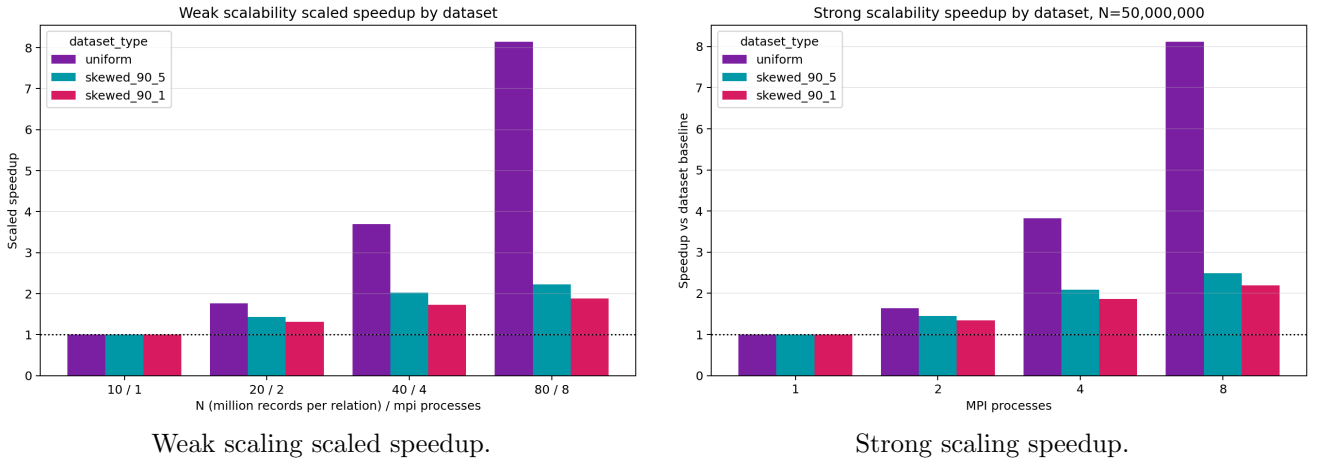


Figure 3: MPI scalability on 1, 2, 4, and 8 nodes/ranks. The left plot reports weak scaling with input size growing with the rank count, while the right plot reports strong scaling with fixed $50M + 50M$ input.

5 Correctness Validation

Correctness is validated at two levels. For small inputs, each implementation can compare its optimized output against a naive $O(N_R N_S)$ reference verifier. For benchmark-size inputs, where the naive verifier is

not practical, the checker compares the final aggregated `join_count`, `checksum1`, and `checksum2` across repeated runs and compatible implementations.

The MPI and hybrid versions compute local values on each rank and then use `MPI_Reduce` to obtain the same three global quantities produced by the sequential baseline. The validation is implemented outside the measured region (not in the executable variants themselves) but in the `checker.cpp` file, so its overhead is not included in the reported runtimes.

6 Conclusions

This module shows that the best execution model depends strongly on the workload and on the target scale. The shared-memory OpenMP version remains the most effective choice for medium-sized inputs that fit comfortably on one node, especially when the dataset is uniform. In this case, the workload is already well balanced and avoiding MPI communication is more important than distributing the data.

The pure MPI version becomes more interesting when the input size grows or when the goal is to exploit multiple nodes. With many ranks and few or no threads per rank, the input data, memory footprint, and memory-bandwidth pressure are distributed across the cluster. This configuration works particularly well on uniform data, where partitions have similar sizes and the cost of redistribution is compensated by the reduction of local work.

The main limitation of pure MPI appears on skewed datasets. Since each complete partition is assigned to one rank, hot partitions can create imbalance: some ranks receive much more work than others. Cyclic ownership reduces this problem by spreading partition ids across ranks, but it cannot split a single very expensive partition.

The hybrid MPI+OpenMP version is useful when each rank receives enough local work to justify using several threads. This is especially true for skewed workloads, where hot partitions make the local join phase expensive. In this case, fewer MPI ranks with more OpenMP threads per rank can be preferable, because OpenMP can parallelize the heavy local work without introducing additional inter-node communication.